Области видимости

Участок: Базовые механизмы прав

Ограничения этого типа работают как фильтр. Они позволяют устанавливать различным группам пользователей доступ к некоторому подмножеству объектов из общего множества.

Области видимости создаются прикладными разработчиками в зависимости от бизнес-требований конкретных участков системы.

Примеры созданных областей видимости:

- по подразделениям/сотрудникам
- по организациям
- по участкам работ
- по складам
- по сумме документа
- отчетность в госорганы

Пользователь без ограничений по организации будет видеть реестр так:

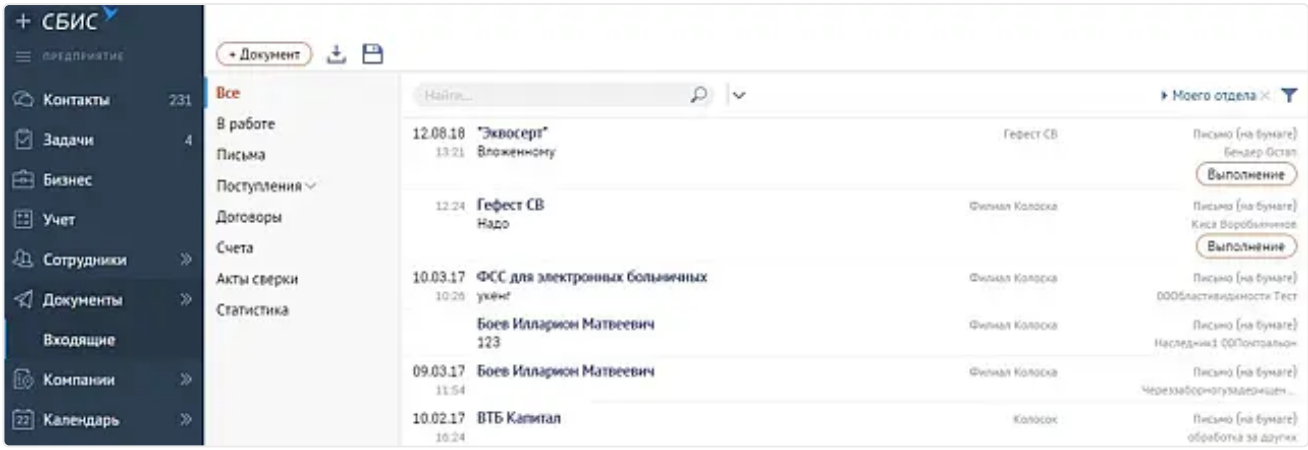

Рис. 1. Реестр входящих писем для пользователя без ограничений.

Пользователь, для которого установлена область видимости по организации "Гефест СВ, ООО", будет видеть реестр иначе:

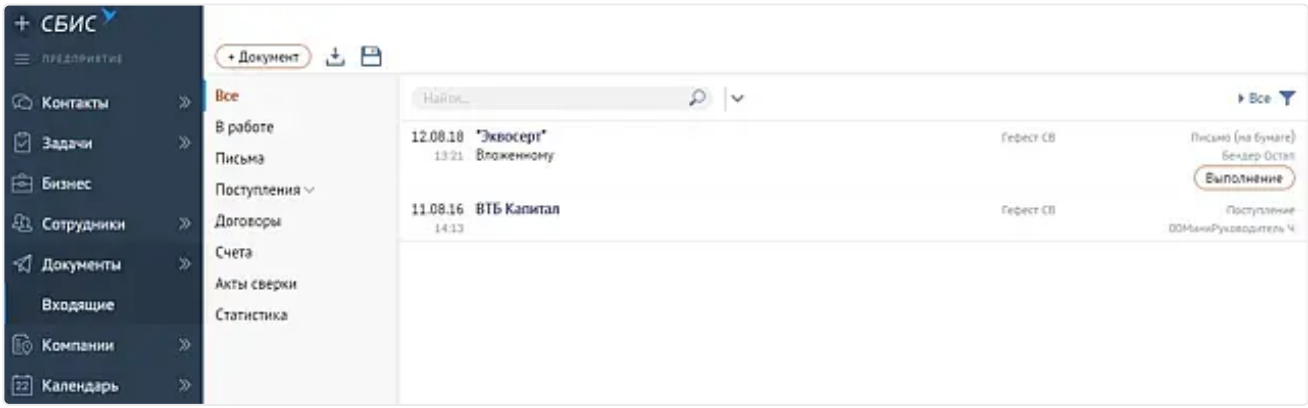

Рис. 2. Реестр входящих писем для пользователя с ограничением по организации "Гефест СВ, ООО".

Создание области видимости

Шаг 1. Настройка ограничения

Добавьте метод бизнес-логики Access area и заполните необходимые поля:

Название	Описание	Пример
Name	Имя метода	ДокументыМоегоОфиса
Display name	Имя, используемое для отображения области видимости в интерфейсе приложения	ОбластьВидимости.ДокументыМоегоОфиса
Display hint	Описание ограничения, выводимое в интерфейсе клиента	Ограничение по назначенному подразделению/офису
Display name for list	Название ограничения для списков	По подразделению/офису
Display name for adding	Название ограничения для окна добавления "Можно детализировать для"	Подразделений
Display order	Определяет порядок отображения ограничения на интерфейсе. Если флаг "Set value": • сброшен, то значение редактора null, • взведен, то в редактор устанавливается значение по умолчанию.	0
Identifier	Строковый идентификатор ограничения Используется в качестве имени переменной транзакции для передачи значений ограничения в SQL-код, а также в качестве имени поля в записи Session.Rights() для получения ограничений текущего метода. Его можно передавать в любые методы системы прав, где требуется ограничения (например, CheckRights.AccessAreaRestrictions(AccessArea='Дела', Restriction='myOfficeDocs')). Должен быть уникальным в рамках сервиса.	myOfficeDocs
Permission policy	Ограничительная политика: • **Allow** - разрешающая. Значение 'Без ограничений' является определяющим при слиянии настроек по участкам системы. • **Deny** - запрещающая. Значение 'Ограничено' является определяющим при слиянии настроек по участкам системы. • **Exclude** - политика с исключениями. Позволяет задать список разрешенных объектов, кроме вложенных. Для этой политики требуется реализация интерфейса IConstraintValue.	Allow
Aos type	Тип ограничения: • **Access area** (Область видимости) — позволяет различным группам пользователей устанавливать подмножество объектов из общего множества, к которому разрешен определенный доступ. • **Working with records** (Работа над записями) — позволяет	Access area

	устанавливать ограничение на запись вида 'разрешено/запрещено'. Такие ограничения подразделяются на: ○ Платформенные — ограничения, которые поставляются с SBIS SDK (удаление, объединение записей, выделение цветом и пр.). ○ Прикладные — создаются разработчиками.	
Display value names source	Метод для получения списка имен сущностей области видимости, например списка организаций	Department.List
Read	Ограничение применяется к методам readonly	true
Modify	Ограничение применяется к методам изменения данных	true
UI editors module	Путь до интерфейсного модуля с прикладным редактором типа для корректного отображения ограничения	Staff/Constraints/user
Available general setting	• **true** - ограничение может быть назначено на исполнителе роли или карточке сотрудника. • **false** - ограничение настраивается только на участке системы.	true
Find role executor	Метод для определения исполнителя регламента по этому ограничению. Если ограничение участвует в алгоритме определения исполнителя для этапа регламента.	Staff.FindRoleExecutorByDepartments
Limited by default	Ограничение значения по умолчанию. Устанавливается, если для области видимости при назначении роли, исполнителю сразу должно заполняться значение.	true
Default value	Значение по умолчанию, если установлен флаг **Limited by default**.	Свой офис
Parameters	Набор вычисляемых предикатов для ограничения, например, "Свой офис", "Свое подразделение".	- 0: Свой офис - 1: Свое подразделение
Return value	Тип данных, возвращаемых методом.	String

Для вычисления предикатов своего ограничения прикладной разработчик должен реализовать свой метод и описать его на вкладке **Code**.

```
def ОбластьВидимости.ДокументыМоегоОфиса(User, Params):
    from Employee.Rights.myOfficeDocs import get_restrictions
    return get_restrictions( User, Params)
```

Пример с указанием ограничительной политики.

```
<object last_changed="Абрамов В.И." last_changed_uuid="79ecbfd8-c7de-446e-982a-429b787d2b2c" name="Constraint" responsible="Кузьмин Н.М.">
```

```
    <comment>Правовые ограничения</comment>
    <aos_function alias="По подразделению/офису"
find_role_executor="Staff.FindRoleExecutorByDepartments"
                  is_service="1"
                  last_changed="Абрамов В.И."
                  last_changed_uuid="79ecbfd8-c7de-446e-982a-429b787d2b2c"
                  limit_by_default="true"
                  name="Constraint.MyOfficeDocs"
                  name_for_adding="Подразделения"
                  name_for_list="По подразделениям"
                  order="300"
                  permission_policy="exclude"
                  responsible="Кузьмин Н.М."
                  responsible_uuid="92e5f6dd-74b0-482e-850a-3cbd919c78c3"
                  return="INT4[]"
                  source_names_vals="Constraint.MyOfficeDocsNamesList"
                  transactionVariableName="myOfficeDocs"
                  type="PYTHON"
ui_editor_module="Staff/Constraints/userAndDepartments">
```

Шаг 2: Создайте функцию области видимости

Пример области видимости "ДокументыМоегоОфиса".

Шаг 3. Настройте интерфейс нового ограничения

Для вывода в интерфейсе редактирования ограничений используется платформенный контрол Property Grid.

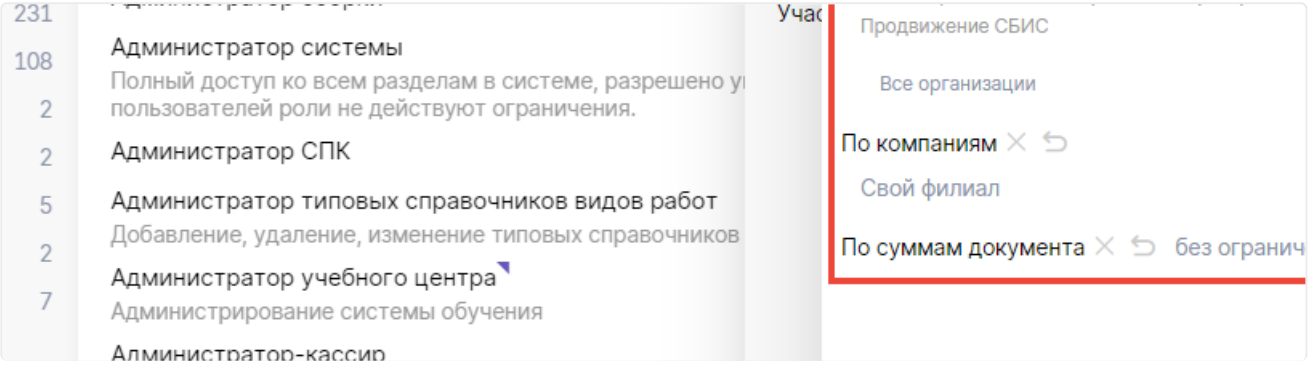

Рис. 3. Настройка PropertyGrid

По умолчанию платформа предоставляет интерфейс редактора только для простых ограничений без выбора значений из справочника.

Если для прикладного ограничения список доступных значений набирается из справочника, должна отображаться визитка выбранной сущности или ограничение требует дополнительных настроек (например, подограничения), то необходимо объявить собственный редактор типа.

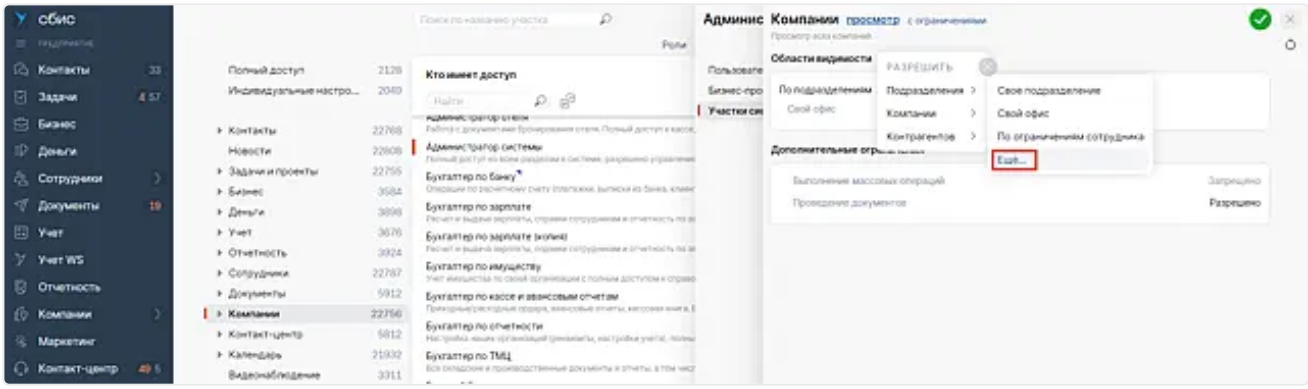

Рис. 4. Объявление собственного редактора типа

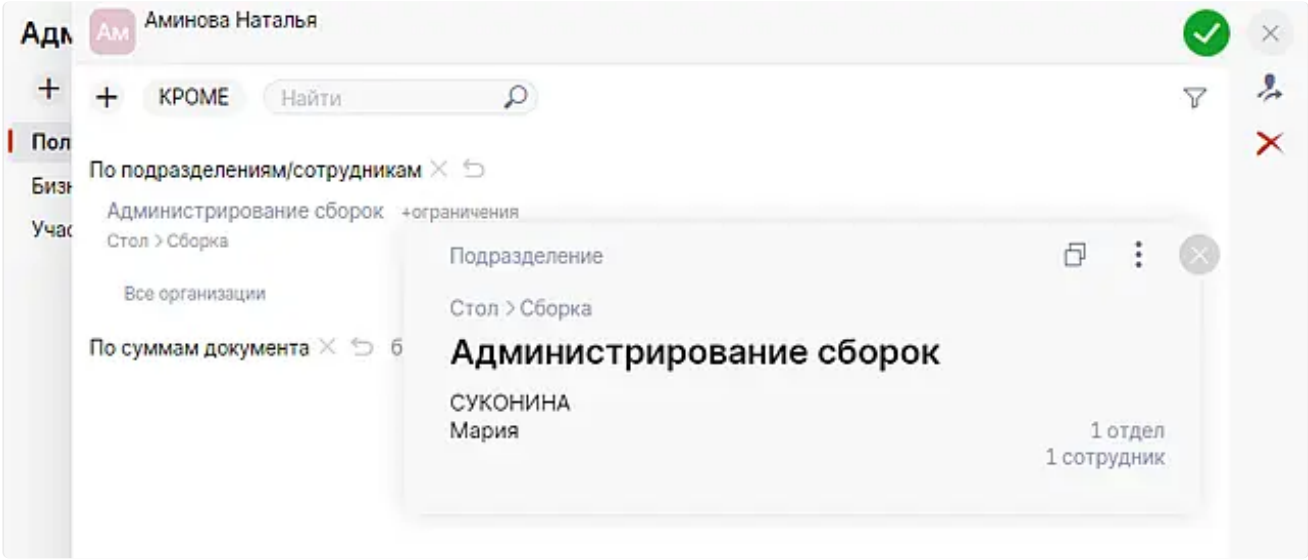

Рис. 5. Пример ограничения выбора из справочника

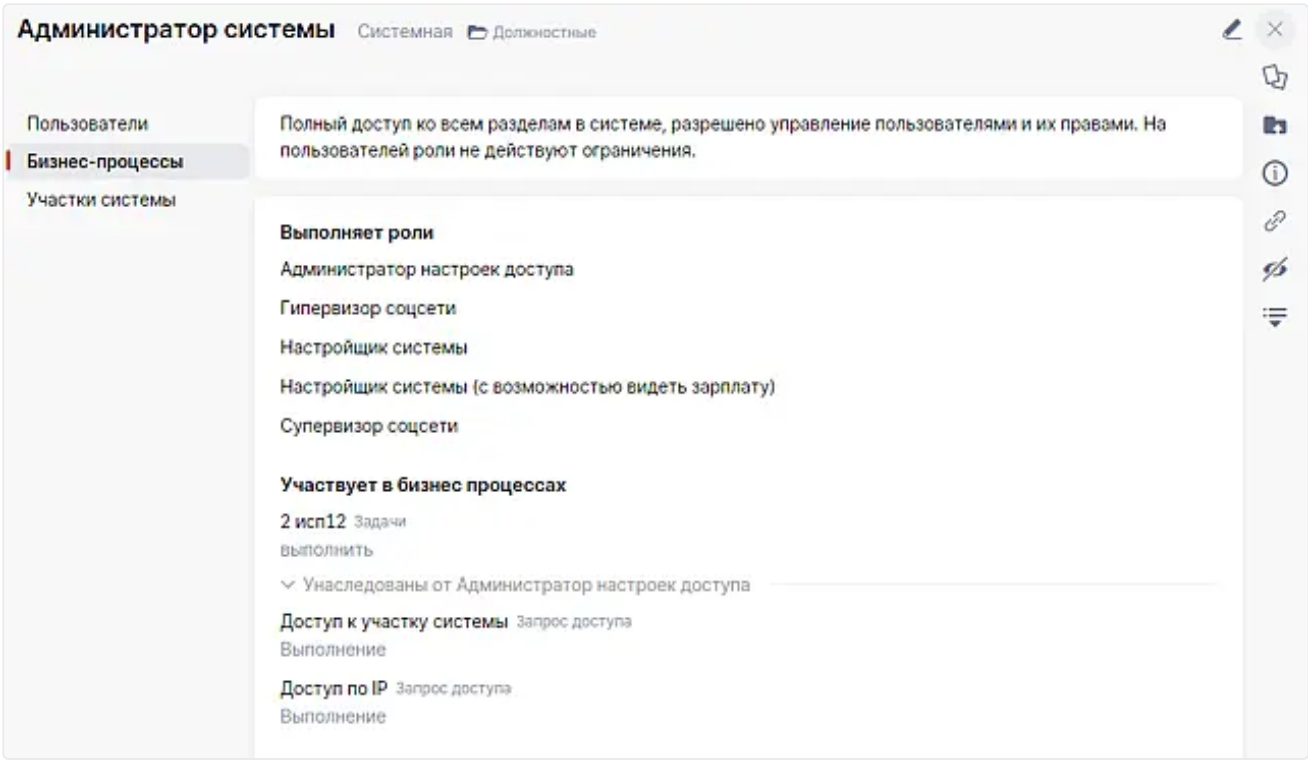

Рис. 6. Пример визитки назначенного ограничения

Пример нестандартного шаблона ограничения (настройка подограничения)

Создайте шаблон в прикладном репозитории своего модуля для отображения ограничения.

Пример шаблона отображения ограничения.

```
<!--ConstraintEditor.wml-->
```

```
<ws:if data="{{_options.realId}}">
    <ws:if data="{{ _options.constraintId === 'ОбластьВидимости.ByUser' }}">
        <!--   значение ограничения "По конкретному сотруднику"  -->
        <Controls.Container.Async templateName="Person/card:Opener" attr:class="staff-constraints__departments_value">
            <ws:partial
                template="{{content.resolvedTemplate}}"
                showMode="hover">
                <ws:miniCardOptions
                    PrivatePersonID="{{ _options.realId }}"
                    privatePerson="{{ _options.realId }}"
                    key="{{ _options.realId }}"/>
                <ws:content>
                    <span class="staff-constraints__departments_caption" on:click="_personClickHandler(_options.realId)">
                        <Controls.decorator:Highlight
                            value="{{ _options.displayName }}"
                            highlightedValue="{{ _options.searchValue }}" />
                    </span>
                </ws:content>
            </ws:partial>
        </Controls.Container.Async>
    </ws:if>
    <ws:else data="{{ _options.constraintId === 'ОбластьВидимости.ДокументыМоегоОфиса' }}">
        <!--   значение ограничения "По подразделению"  -->
        <Controls.Container.Async templateName="Department/Opener" attr:class="staff-constraints__departments_value">
            <ws:partial
                template="{{content.resolvedTemplate}}"
                showMode="hover">
                <ws:miniCardOptions oid="{{ _options.realId }}"/>
                <ws:content>
                <span class="staff-constraints__departments_caption" on:click="_openDepartmentCardHandler(_options.realId)">
                    <Controls.decorator:Highlight
                        value="{{ _options.displayName }}"
                        highlightedValue="{{ _options.searchValue }}" />
                </span>
                </ws:content>
            </ws:partial>
        </Controls.Container.Async>
    </ws:else>
</ws:if>
<ws:else>
    <Controls.decorator:Highlight
        value="{{ _options.displayName }}"
        highlightedValue="{{ _options.searchValue }}" />
</ws:else>
```

В прикладном репозитории своего модуля создайте библиотеку, которая по умолчанию экспортирует объект, поля которого соответствуют интерфейсу IConstraintView.

Интерфейс IConstraintView

- **name** — идентификатор ограничения. Берется из Genie, смотрится в uax.
- **editorTemplateName** — имя контрола, который будет использоваться в качестве редактора для ограничения с типом "Область видимости". Контрол должен соответствовать интерфейсу IConstraintEditor.
- **editorOptions** — опции котнрола, который задан в опции editorTemplateName.

```
/**
 * Staff/Constraints/user
 * Библиотека для работы с ограничением "По сотруднику"
 * @library
 * @author Курдамосов С.А.
 */

export default {
    // Идентификатор ограничения
    name: 'ОбластьВидимости.ByUser',
    // Имя платформенного контрола-редактора
    editorTemplateName: 'Rights/constraintEditor:ConstraintEditor',
    // Настройки прикладного редактора выбора значений
    editorOptions: {
          // Путь до шаблона ограничения
          itemTemplate: 'Staff/Constraints/ConstraintEditor',
          displayProperty: 'Лицо',
          //Интерфейс справочника выбора значений
          selectorTemplate: {
              templateName: 'Staff/Choice/List',
              templateOptions: {
                  useCustomScroll: true,
                  showContragentSwither: true,
                  showOtherEmployees: true,
                  allowEnterToFolder: false,
                  showWorkGroups: false,
                  multiSelect: true
            }
          }
      }
    }
};
```

Интерфейс IConstraintValue

Реализация интерфейса IConstraintValue требуется для ограничений с политикой Exclude. Функция области видимости должна иметь общую реализацию с IConstraintValue и их поведение не должно различаться.

```
class IConstraintValueBase
{
    public:
    virtual ~IConstraintValueBase(){}
}

template< typename T >
class IConstraintValue : public IConstraintValueBase
{
```

```
public:
    // Получить все множество доступных объектов с разворотом
    virtual OptArray< T > GetAllAccessibleIds() = 0;
    // Получить множество доступных объектов без разворота
    virtual boost::optional< Record > GetAccessibleSet( bool resolve_flags = true ) = 0;
    // Получить по массиву идентификаторов список разрешенных объектов
    virtual OptArray< T > GetAccessibleIds( OptArray< T > const& ) = 0;
    // Доступен ли конкретный объект
    virtual bool IsObjectAccessible( T const& id ) = 0;
}
```

Число методов в реализуемом классе может быть больше, если это требуется для ограничения. В примере указаны методы, которые должны быть в классе обязательно.

Прикладной разработчик для своего ограничения делает собственную реализацию на C++:

```
class MyConstraintValue: public IConstraintValue< T >
```

Также прикладные реализации поддерживаются на Python с аналогичной сигнатурой:

```
object IConstraintValue(object):
    def GetAllAccessibleIds(self) -> list:
        """Получить все множество доступных объектов с разворотом"""
        pass

    def GetAccessibleSet(self, resolve_flags: bool = False) -> sbis.Record or None:
        """Получить множество доступных объектов без разворота"""
        pass

    def GetAccessibleIds(self, ids: list) -> list:
        """Получить по массиву идентификаторов список разрешенных объектов"""
        pass

    def IsObjectAccessible(self, object_id: object) -> bool:
        """Доступен ли конкретный объект"""
        pass
```

Пример IConstraintValue приведен здесь.

Для доступа из одного языка к ограничению, реализованном на другом языке, используются классы обертки PythonConstraintValue и CppConstraintValue.

В момент загрузки модуля в обработчике OnEndAllLoadModules устанавливается соответствие между ограничением и конкретной реализацией IConstraintValue с помощью вызова PermissionManager::RegisterConstraint.

Для Python принимается класс значения ограничения, для создания конкретного значения вызывается конструктор указанного класса с передачей идентификатора пользователя и значения ограничения в виде Record (те же аргументы, что сейчас передаются в функцию области видимости).

```
def register_constraint():
    """Зарегистрировать StaffConstraintValue"""

permission.PermissionManager.Instance().RegisterConstraint('myOfficeDocs', StaffConstraintValue)
```

Для C++ в RegisterConstraint принимается функтор, который по тем же аргументам должен вернуть std::unique_ptr< IConstraintValue< T > >.

```
template< typename T >
void RegisterConstraint( StringView const constraint_id,
std::shared_ptr< ConstraintValueCreator< T > > const& creator )
{
   RegisterConstraintImpl( constraint_id, creator );
}
```

В рамках одного языка для одного ограничения можно зарегистрировать только одну реализацию. Последующие вызовы будут перекрывать ранее выполненную реализацию. Для ограничения может быть зарегистрирована одна реализация на каждом поддерживаемом языке.

При создании значения ограничения используется реализация текущего языка, если такая существует. Если на текущем языке реализации нет, то используется реализация на другом языке через обертки PythonConstraintValue и CppConstraintValue, т.е. используется "ближайшая" для потребителя реализация. Также это позволяет расширить интерфейс IConstraintValue для конкретного ограничения, если в этом будет необходимость.

Интерфейс IConstraintEditor

- **itemTemplate** — шаблон отражения названия ограничения.
- **displayProperty** — имя поля, в котором содержаться название ограничения. Используется при выборе значения из справочника для записи в историю выбора.
- **selectorTemplate** — настройка справочника выбора.
  - **templateName** — шаблон панели выбора элементов.
  - **templateOptions** — параметры шаблона всплывающего окна. Свойства шаблона всплывающего окна задаются прикладным разработчиком.
  - **footerTemplate** — шаблон подвала выбранного значения ограничения.
  - **placeholder** — шаблоны для поля placeholder. Задаются, как объект вида ключ - шаблон. По умолчанию шаблон суммы.
  - **placeholderKeyCallback** — функция обратного вызова для получения идентификатора шаблона.

Шаг 4. Создайте метод определения исполнителя регламента

Если область видимости участвует в алгоритме определения исполнителя, то для нее в том же модуле где реализовано ограничение должен быть определен метод FindRoleExecutor.

Например, для ограничения "По сотрудникам компании".

Staff.FindRoleExecutorByDepartments(params, executors) : Int64

Сервис: online

Модуль: Прикладной модуль, где описана функция области видимости

Доступ: Internal

Входные параметры:

- **params:** sbis.Record — запись, содержит критерии фильтрации исполнителей по параметрам документа
  - **face_id:** int64 — идентификатор исполнителя документа
  - **dept_id:** int64 — идентификатор подразделения документа
  - **org_id:** int64 — идентификатор организации
  - **work_areas:** array_int64 — идентификаторы зон ответственности
  - **sum:** money — сумма документа
- **executors:** sbis.RecordSet — набор отсортированных наборов исполнителей роли в порядке от наиболее приоритетного к менее приоритетному:
  - **Executors:** sbis.RecordSet — набор записей по исполнителям:
    - **ExecutorId:** int64 — идентификатор (@Лицо) исполнителя
    - **Constraint:** sbis.RecordSet — значения ограничения
      - **Id:** — идентификатор правового значения, тип определяется возвращаемым значением функции области видимости

- **VariablePosition:** — номер абстрактного значения , совпадает с номером, отдаваемым в функцию области видимости, например, для ограничения по подразделениям:
  - 0 — свое подразделение,
  - 1 — свой офис. Если задано VariablePosition, то поле Id не заполнено.
- **Min:** Money — минимальная сумма документа
- **Max:** Money — максимальная сумма документа
- **AdditionalId:** Int64 — лицо (сотрудник/подразделение/организация), для которого указано ограничение.